

Fundamentos de la programación 2

Práctica 1. UnblockMe

Indicaciones generales:

- La línea 1 del programa y siguientes deben contener los nombres de los alumnos de la forma:
`// Nombre Apellido1 Apellido2`
- **Lee atentamente** el enunciado e implementa el programa tal como se pide, con los métodos, parámetros y requisitos que se especifican. No puede modificarse la representación propuesta ni alterar los parámetros de los métodos pedidos, a excepción del modo de paso de dichos parámetros (out, ref,...), que debe determinar el alumno. Pueden implementarse todos los métodos adicionales que se consideren oportunos, especificando claramente su cometido, parámetros, etc.
- El programa, además de correcto, debe estar bien estructurado y comentado. Se valorarán la claridad, la concisión y la eficiencia.
- La entrega se realizará a través del campus virtual, subiendo únicamente el archivo `Program.cs`, con el programa completo.
- El **plazo de entrega** finaliza el 3 de marzo. Puede entregarse antes de esa fecha.

Vamos a implementar el conocido puzzle *UnblockMe* que puede encontrarse como app para móvil y también online. Consta de un conjunto de bloques sobre una cuadrícula de $n \times m$ celdas, con una apertura en uno de los lados. Los bloques pueden ser horizontales, de tamaño $1 \times k$, o verticales, de tamaño $k \times 1$, siempre con $k > 1$. Cada bloque puede moverse en la dirección de su eje, siempre dentro de la cuadrícula y sin empujar a otros bloques. El juego consiste en sacar de la cuadrícula un bloque concreto por la apertura dada.¹

A continuación mostramos distintas representaciones de una misma configuración del juego:



Fig. 1



Fig. 2

#####	level 1
#...a..#	b
#...a..#	...a..
#bb.a...	...a..
#.....#	bb.a..
#c.ddd.#	
#c..ee.#	c.ddd.
#####	c..ee.

Fig. 3

Fig. 4

En Fig. 1 se muestra el aspecto del juego original y en Fig. 2 el aspecto que tendrá nuestra versión en consola. En este nivel hay que sacar el *bloque objetivo* (rojo en la versión original, verde en nuestro caso) por la apertura del muro de la derecha.

Para implementar el juego utilizaremos la siguiente **representación interna**:

```
// coordenadas (x,y) para representar posiciones y direcciones de desplazamiento
struct Coor {
    public int x,y;
}

struct Estado{ // estado del juego
    public char[,] mat; // '#' muro; '.' libre; letras 'a','b' ... bloques
    public char obj; // char correspondiente al bloque objetivo (el que hay que sacar)
    public Coor act, sal; // posiciones del cursor y de la salida
    public bool sel; // idica si hay bloque seleccionado para mover o no
}
```

¹Para comprender mejor la mecánica del juego, puede jugarse online en <https://www.quickflashgames.com/games/unblock/> y también en <https://scratch.mit.edu/projects/200444673/>

El tipo **Coor** permitirá representar posiciones $(fila, columna)$ de la cuadrícula, así como vectores de dirección para los desplazamientos de bloques. La posición $(0,0)$ corresponde a la esquina superior izquierda. Las direcciones abajo, arriba, izquierda y derecha se representan como vectores $(1,0)$, $(-1,0)$, $(0,-1)$ y $(0,1)$, respectivamente.

El tipo **Estado** representa la información del juego en un instante dado:

- **mat** es una matriz de caracteres que representa la cuadrícula y los bloques. Tendrá dimensiones $(filas+2, columnas+2)$, i.e., contiene 2 filas y 2 columnas extra para representar los bordes, que corresponden al marco blanco de Fig. 2. Esta representación es frecuente en tableros para simplificar las comprobaciones al mover los elementos del juego. Internamente representaremos los muros con el carácter '#', las posiciones libres con '.' y cada bloque con un carácter desde 'a' en adelante. En Fig. 3 aparece la matriz de caracteres **mat** correspondiente a Fig. 1 y Fig. 2.
- **obj** es la letra que indica el bloque objetivo que hay que sacar de la cuadrícula. En el ejemplo de arriba es el bloque verde, que corresponde a la letra 'b' en nuestra representación del juego, como veremos más adelante.
- **act**, **sal** denotan la posición actual del cursor y la de la casilla de salida, respectivamente. En nuestro ejemplo sera **act**=(2,2) y **sal**=(3,7).
- **sel** indica si hay bloque seleccionado para mover (**true**) o no (**false**).

La **mecánica del juego** consiste en mover bloques para despejar la salida del bloque objetivo y sacarlo por la apertura dada. El cursor puede moverse libremente por la cuadrícula con las teclas de dirección. Cuando está sobre un bloque se puede seleccionar dicho bloque con la tecla "Space" y a continuación moverlo con las teclas de dirección. Pulsando nuevamente la tecla "Space" se deselecciona el bloque y el cursor vuelve a moverse libremente. Cuando no hay bloque seleccionado en el estado tendremos **sel=false**) y el cursor se mostrará en pantalla como '**'. Cuando el cursor está sobre un bloque y está seleccionado tendremos **sel=true** y el cursor se mostrará en pantalla como '<'. Los movimientos del cursor arrastrarán dicho bloque siempre que tenga hueco para el movimiento en la dirección dada.

Implementaremos los siguientes métodos (es recomendable hacerlo incrementalmente en el orden propuesto, comprobando el buen funcionamiento de cada uno de ellos):

- **Estado LeeNivel(string file, int n)**: abre el archivo indicado en **file**, busca el nivel **n** y devuelve un **Estado** con dicho nivel. El archivo proporcionado **levels.txt** contiene unos pocos niveles de juego en formato textual. Cada nivel, como aparece en Fig. 4 está encabezado por una línea de la forma **level n** (siendo **n** un entero concreto); a continuación hay un línea con un solo carácter correspondiente al *bloque objetivo* y después viene la matriz propiamente dicha con '.' para los huecos y caracteres para los bloques. El final de un nivel viene determinado por una o más líneas en blanco.

Este método comenzará buscando el principio del nivel y a continuación leerá la información de dicho nivel, generando un **Estado** que devolverá como resultado. Debe crear los bordes tal como se ha explicado y colocar el cursor en la posición $(1,1)$.

- **void Render(Estado est)**: muestra en pantalla el estado del juego, en el formato de Fig. 2, con un color para cada bloque. Podemos obtener el vector de colores disponibles para consola como:

```
ConsoleColor[] colores = (ConsoleColor[]) ConsoleColor.GetValues(typeof(ConsoleColor));
```

Para los huecos utilizaremos el negro, que corresponde al primer elemento de **colores** y para los muros el blanco, que corresponde al último elemento del vector. Una forma sencilla de asociar los caracteres de los bloques a índices del vector es utilizar el siguiente método (que ya excluye el negro):

```
// 'a' -> 1, 'b' -> 2... descartar el 0=negro
static int BloqueToInt(char c){
    return ((int) c) - ((int) 'a') + 1;
}
```

Tal como se muestra en Fig. 2, cada uno de elementos de la matriz, en pantalla se representará con dos caracteres, para obtener una representación más proporcionada del juego. Para el cursor, si hay bloque seleccionado escribiremos '<>' y '**' en otro caso. Después de la matriz escribiremos el color del bloque objetivo para identificar dicho bloque en la matriz.

- **void MarcaSalida(Estado est):** dado un estado del juego, localiza el bloque objetivo y la dirección del mismo, y entonces busca y marca la salida correspondiente. Si el bloque objetivo es horizontal marcará la salida en el muro derecho, en la misma fila; si es vertical la salida estará en el muro inferior, en la misma columna.
- **void MueveCursor(Estado est, Coor dir):** dado el estado **est**, si no hay bloque seleccionado para mover, mueve el cursor en la dirección **dir**, siempre que no tope con un muro y ni con la posición de salida.
- **void MueveBloque(Estado est, Coor dir):** dado el estado **est**, si el cursor está en una posición de bloque y en modo desplazamiento, mueve dicho bloque en la dirección **dir**, siempre que no tope con un muro u otro bloque. Para ello será útil un método auxiliar **BuscaCabeza** que determine la posición más avanzada (cabeza) del bloque a mover, en la dirección **dir**. Una forma simple de mover un bloque en nuestra representación consiste en ocupar la posición siguiente a la cabeza y borrar la posición de la cola. Al mover un bloque en una dirección, el cursor también se moverá en la misma dirección.
- **void ProcesaInput(Estado est, char c):** actualiza el estado del juego **est** de acuerdo al input **c** que recibe como parámetro. El comportamiento en función del carácter **c** es el siguiente:
 - **'s':** si el cursor está sobre un bloque cambia el modo *bloque seleccionado* (invierte el valor del campo booleano **sel**); si no está sobre un bloque, no hace nada.
 - **'u' 'd' 'l' 'r':** determinan la dirección del movimiento del cursor (arriba, abajo, izquierda, derecha) y con esa dirección:
 - si hay bloque seleccionado, se mueve con el método **MueveBloque**
 - en otro caso, se mueve el cursor con **MueveCursor**

El input **'z'** por ahora no hace nada (se utilizará más adelante para deshacer jugada) y **'q'** se procesará desde el método **Main** y sirve para terminar el juego.

- **char LeeInput():** lee un input de teclado y devuelve un carácter que representa dicho input y se da ya implementado:

```
static char LeeInput() {
    char d = ' ';
    while (d == ' ') {
        if (Console.KeyAvailable) {
            string tecla = Console.ReadKey ().Key.ToString ();
            switch (tecla) {
                case "LeftArrow": d = 'l'; break; // direcciones
                case "UpArrow": d = 'u'; break;
                case "RightArrow": d = 'r'; break;
                case "DownArrow": d = 'd'; break;
                case "Delete": d = 'z'; break; // deshacer jugada
                case "Escape": d = 'q'; break; // salir
                case "Spacebar": d = 's'; break; // selección de bloque
            }
        }
    }
    return d;
}
```

- **void Main():** solicita un nivel de juego, lo lee del archivo **levels.txt** para generar el estado de juego, marca la posición salida y renderiza en pantalla.

A continuación arranca el *bucle principal* que hace las siguientes acciones **por este orden**: lee un input de teclado, lo procesa y muestra el estado. El bucle termina cuando el bloque objetivo alcanza la posición de salida o bien el usuario aborta el juego (con la tecla "Escape", que **LeeInput** devuelve como **'q'**).

Con estos métodos tendremos una versión completamente funcional del juego, que debe probarse bien antes de seguir adelante. A continuación extenderemos la funcionalidad del programa:

- **Deshacer jugadas.** Permitiremos que el usuario pueda deshacer movimientos y volver a un estado anterior mediante la tecla “Delete” para la que `LeeInput` devuelve ‘z’. Por “movimiento” entendemos *movimiento de bloque*, puesto que los desplazamientos del cursor que no mueven bloques, no suponen un cambio representativo en el estado del juego. Un movimiento puede determinarse con un pareja (*Posición del cursor, Dirección*) y para deshacer dicho movimiento basta con volver a aplicar el mismo movimiento, pero con dirección opuesta.

Para poder deshacer jugadas necesitamos llevar memoria de los movimientos realizados. Una forma natural es mantener un *array de jugadas* donde iremos almacenando cada una de ellas de izquierda a derecha. Mantendremos además un índice a la primera posición libre, que permite almacenar nuevas jugadas, así como deshacer las mismas.

Debemos implementar un tipo struct `Jugada` para los movimientos y otro struct `Memoria` con el array de jugadas y el índice que acabamos de mencionar. Declararemos una variable `memoria` en `Main` y la pasaremos a los métodos correspondientes para gestionar la memoria de jugadas y la opción de deshacer.

- **Archivo de records.** Gestionar un archivo de record de movimientos para cada nivel que lleve cuenta del mínimo número de movimientos utilizado para cada nivel. Inicialmente el archivo estará vacío. Cada vez que se acaba un nivel se busca en el archivo el record para ese nivel. Si no existe, se crea una nueva entrada; si existe se compara con el actual y se guarda el menor, informando al usuario del nuevo record.
- **OPCIONAL Diseñador de niveles.** Implementar un método para generar un nuevo nivel del juego. Para ello se parte de una disposición de bloques en forma resuelta, i.e., con el bloque objetivo en la posición de salida. A partir de ahí se pueden generar movimientos aleatorios (o guiados) para “descolocar” los bloques. Esta forma de trabajo garantiza que cualquier disposición obtenida tenga solución, pero generar puzzles *interesantes* con este método no es trivial.